Supervising a Generalist Robot Policy Through a Rule-Bound, Long-Horizon Relay: A Harness, Two Simulators, and What the Teleoperation Data Got Wrong

Darsh Patel* Kaushik Muhammad Mukthar

Physical AI Hacks (MakerMods × Dream Labs), 7–8 November 2026

Preprint, 9 November 2026

Abstract

We report the system we built over a 36-hour hackathon to run a generalist vision–language–action (VLA) policy, Dream Labs' Dream-1, through a timed three-stage relay on two brakeless 7-motor MakerMods Metal arms: unplug USB cables from a hub, stack a tower above 4 in, and drop legal golf balls into a bin while avoiding an illegal ball. The rubric turns ordinary failures into match losses: touching a later stage's object before the current pass bar is a disqualification (DQ), and binning the illegal ball loses the match. The policy is a black box behind an HTTP station agent, so our only inputs are discrete: which prompt to send, and when to stop, home or retry. We contribute (i) a supervisory harness that treats the relay as a stage state machine with two-look verification, a 10 Hz local illegal-ball guard, sensor overrides and template-only prompting, together with a formal decision model for tuning it; (ii) two simulators, a virtual-clock stress simulator that plays 37 scenarios whose faults reuse the error types logged on the station, and a MuJoCo table driven by the real Dream-1 API with a ground-truth referee; (iii) a teleoperation pipeline for the Metal arms (a gravity-compensated leader/follower and a Quest headset path with a wrist-flip guard); and (iv) a diagnosis of why our fine-tune failed. Measured on the station, the median policy round trip was 1.12 s and 67% of 1,650 action chunks ran in stop-and-wait mode, leaving the arms idle about 30% of the time. In 185 simulated matches per configuration, a faster judge raised mean points from 61.2 to 70.5 but lost 28 matches to the illegal ball; adding the local guard kept 69.8 points and lost none. With the real policy in MuJoCo, base Dream-1 unplugged a cable in 14 of 25 matches but completed 0 of 54 put-downs. The fine-tune was worse than the base model on every task; its Quest demonstrations carried a 5.5 cm gravity-sag offset between commanded and measured tool pose that the leader-arm data did not. We have no logged per-stage success rates from the physical station, and we say so wherever it matters.

1 Introduction

Generalist VLA policies [1–3] take a camera stream and a sentence and return a chunk of actions. They are increasingly offered as a service: the team holds an API key, a station agent on the robot computer runs the control loop, and the model runs elsewhere. This changes what a team can engineer. Within a hackathon you cannot retrain the base model or add a controller underneath it. You can choose the words, decide when a subtask is finished, stop the arms, and collect demonstrations for a fine-tune.

The Physical AI Hacks "Hill Climb" relay made those choices consequential. Each team got a station with two MakerMods Metal arms (seven Damiao motors each, no brakes), an overhead camera, one wrist camera per arm, and Dream-1 through Dream Labs' SDK. A match lasts 5 minutes, and nobody may touch the robot or send input to the policy after "go". Table 1 gives the scoring. Two rules make it unlike a benchmark success rate. First, the stages are gated: a gripper that touches a block before a cable is out is a DQ, and so is touching a ball before the tower stands. Second, two black (on our table, white) balls lie among the legal ones, and binning either loses the match. A harness that declares "done" too early, or one that lets the policy wander, scores zero.

This paper describes what we built and what we measured, in the order the work happened:

*Corresponding author. Code: https://github.com/darshp848/hill-climb-harness, https://github.com/darshp848/task_simulation, https://github.com/darshp848/follower_code.

Table 1: Rubric v4 of the 1v1 Hill Climb Relay (abridged). Pass bars gate the next stage.

Stage	Pass bar	Points
1. Unplug USB cables	≥ 1 cable out of the hub	+10 first, +5 each further; +20 per loose cable plugged in (max 4)
2. Stack	tower > 4 in, free-standing 3 s after release	+20; +3 per full inch; +10 taller tower (head-to-head)
3. Balls	none	+5 per legal ball in bin; illegal ball in bin = match lost
Integrations	shown to referee before the match	+10 LiveKit, +5 Dimensional
DQ: touching a later stage's object before the current pass bar; any human input after "go".		

- **A supervisory harness** (Section 4) that never commands a motor. It runs the relay as a state machine over Dream-1, verifies each stage with two agreeing looks from a vision-language judge plus gripper sensors, guards the illegal ball with a local detector at 10 Hz, and composes every prompt from tested templates. We also write its decision rules as a small set of equations so they can be tuned in simulation rather than by hand (Section 5).
- **Measurements of the serving loop** (Section 6): where the 1.1 s round trip goes and why the arms stand still for about a third of each run.
- **Two simulators** (Section 7) and the results from 185-match sweeps, a 30-configuration judge benchmark, and 25 MuJoCo matches with the real policy (Section 8).
- **Teleoperation and fine-tuning** on the Metal arms (Section 9), including a diagnosis of the fine-tune's failure that traces it to the data-collection path rather than the model.

Section 10 lists what we could not measure. The main gap is plain: the station logged policy calls but not task outcomes, so we report no physical success rate for any stage.

2 Related work

Language-conditioned policies and supervisors. SayCan [4] and Inner Monologue [5] put a language model above a library of skills and feed scene feedback back into the plan; Code as Policies [6] writes the supervisor as code. Our setting is narrower and stricter. There is one skill, a monolithic bimanual VLA, and the interface is a single prompt for both arms. The supervisor must also respect gating rules whose violation is not recoverable.

Success detection. Using vision-language models to judge task success has been studied directly [7], and REFLECT [8] uses them to explain failures. We use a judge for both stage completion and a safety guard, and we find that the guard question is the harder one for most models (Section 8.2).

Teleoperation and data. Leader/follower arms [9, 10] and LeRobot datasets [11] are now the default path to fine-tuning data. Our diagnosis in Section 9.2 is a case where two collection paths on the same arm produced inconsistent state-to-action maps, which the loss curve did not reveal.

Evaluation in simulation. SIMPLER [12] warns that simulated evaluation is useful only when the simulator's noise matches reality. We report the stress simulator's numbers as tests of harness logic only, and we keep them apart from the MuJoCo runs with the real policy. For uncertainty we follow the bootstrap reporting advocated by Agarwal et al. [13].

3 Platform

Hardware. Figure 1 shows the station. Each Metal arm has six joints plus a gripper, driven by Damiao motors over CAN at 1 Mbit/s from an Intel NUC. The station agent declares `body: metal`. Early on, an installed YAM driver library misled us into treating the arms as YAM arms; our leader/follower code now refuses to run unless the station config names the arm type it was written for.



Figure 1: Station C on 7 November. Two MakerMods Metal arms on a rail, the overhead camera on the centre pole, and a wrist camera on each gripper. On the table: the power strip with orange USB cables (stage 1, unplugged from the wall), blue and pink foam cubes and yellow/pink sticks (stage 2), golf balls and the white bin (stage 3). On this table the white golf ball is the illegal one.

Control stack. We read the station's installed SDK (`dreamlabs` 1.0.37). The policy returns 48-row chunks, played at a policy rate of 20 Hz (2.4 s per chunk). A command thread interpolates rows at 250 Hz, but the Metal bus thread reads joint state and sends MIT-mode setpoints at a hard-coded 30 Hz, so seven of every eight interpolated commands never reach a motor. The velocity feed-forward is low-pass filtered with a time constant of about 0.8 s at that tick rate. A client-side pacer runs chunk requests asynchronously (the arm keeps moving while the next chunk is computed) only while the median round trip is under 900 ms, and otherwise falls back to stop-and-wait.

Station API. The agent exposes `/status`, `/start`, `/stop`, `/home`, `/reset` and `/prompt` on localhost, plus a debug view with camera trips. A station add-on we wrote serves single camera frames at 640 px width (about 35 KB) and gripper and joint readings (`/arms`). One prompt drives both arms; there is no per-arm command.

4 The harness

Figure 2 shows the structure. The harness runs on the match laptop, has no motor driver, and talks only to the station agent. Preflight refuses to start if the station is busy, an arm is down, or a camera frame is older than 2 s. An arm fault, a station error, the time budget or an operator stop all send `/stop`, which holds the arms where they are.

4.1 Plans and the stage state machine

A plan is a YAML list of stages. Each stage has a prompt (or a template), a yes/no success question, a timeout, a retry budget, recovery prompts and an `on_fail` policy. The top-level order follows the rubric and the operator's notes: unplug every seated cable, optionally plug loose ones in, build the tower, then balls. Because moving on from a pass-bar stage is a DQ, those stages retry until the clock runs out; only scoring stages after a pass bar use the "two tries, three if worth 20 points or more" rule. Hard cutoffs (for example, leave plug-in by $t = 150$ s) keep high-value but unreliable stages from starving later ones. Retries use `/prompt` on the running session rather than stop and start, which costs 2.9 s (median) before motion resumes.

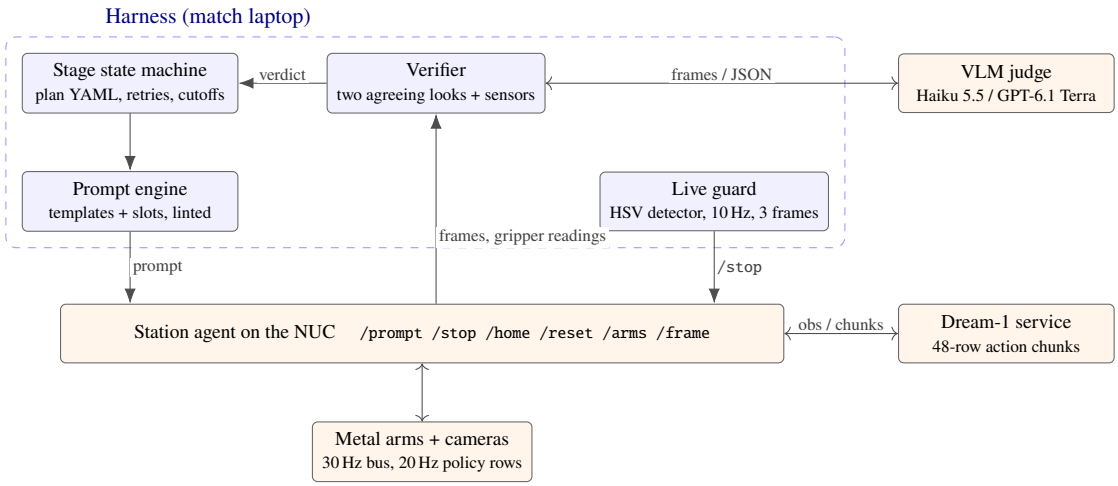


Figure 2: The harness sits beside the station agent. It sends prompts and stop/home commands over local HTTP and reads frames and gripper readings; every motor command, speed cap and clamp stays inside Dream's control loop. The live guard has its own path to `/stop`, independent of the judge.

4.2 Verification

A stage is done only after two agreeing "done" verdicts. The judge receives a stitched grid: the stage-start frame, the last six overhead frames and the newest wrist frames, with detector boxes drawn on them, and returns JSON (`done`, `progress`, `status`, `holding`, `guard`). Two staggered calls are kept in flight, which on the station halved the time between verdicts from about 9.7 s to 4.9 s. Gripper readings from `/arms` overrule the judge's "holding" answer: a gripper closed fully on nothing is not holding a cable, whatever the image seems to show.

4.3 The live guard

The illegal-ball guard cannot wait for a multi-second judge. A local detector (HSV thresholds, blob extraction and a shape test that separates balls, cubes and lying sticks; no learned weights) runs on both wrist frames at 10 Hz in its own thread. Three consecutive positive frames send `/stop` directly, 0.3–0.5 s after the ball enters view. Detection on both wrist views takes 76 ms per poll after vectorisation (236 ms before). The guard also blocks `/reset`, which moves both arms and cannot be cancelled once started.

4.4 Prompting

Every instruction Dream-1 receives comes from a template with slots (object colour, position, the next tower piece, the current top piece), and no model writes free text to the robot. Templates are linted at load: a word cap, no negations, no arm named in an arm-agnostic template, and no unfilled slot. Slots are filled from plan defaults or by one cheap judge call that chooses among numbered options; a refill that produces the same text is not resent. For a fine-tuned checkpoint the plan switches to `verbatim` mode, sending the recorded task texts byte for byte, and the loader refuses any rewriting option alongside it.

These rules came from hand trials on the station with base Dream-1 (Table 2). Three findings shaped them. Naming an arm hurt: "with the right arm" forced the far arm onto blocks at the edge of its reach, while the same request with no arm named was done by the left arm on the first try. Long, coached prompts did worst; a 111-word cable prompt that described approach angle and cable orientation was the worst of the day. And cable prompts are refused without a true safety sentence stating that the strip is unplugged and the cords are insulated low-voltage USB. We stated the facts; we did not try to defeat the filter.

Table 2: Hand-tested prompts on the station (base Dream-1, 8 Nov). Counts are small and were judged by eye.

Task	Prompt (abridged)	Outcome
Unplug	"With the right gripper half open, move down onto the plug . . . close the gripper on the plug and pull it straight up." + harness regrip and pull	held the plug through the pull, 2/2
Unplug	"Remove the first cable on the right from the power strip by pulling its plug straight up, then place it on the right side."	1/3 (best policy-only wording)
Ball	"With the right arm, pick up the orange golf ball on the right side of the table and drop it into the white basket."	in, ~35 s
Stack	"put the pink cube on top of a blue cube" (no arm named)	first try; repeat stacked the wrong cube
Stack	". . . lift it high and lower it gently straight down onto the blue cube, then open the gripper . . . "	pink on blue, ~85 s
Failed	arm named for stacking; whole-tower prompts; two arms in one prompt; "eagle's-eye" approach; direction words for pulling	see text

Table 3: Offline bench of Dream's Metal driver on a fake Damiao bus (0.4 Hz sweep, gripper late half the time). Tracking RMS per joint, degrees.

Setting	Bus tick (Hz)	j0	j1	j2	j3	j4	j5
Stock: 30 Hz, FF τ 0.8 s	30.0	2.24	2.89	3.64	4.46	1.73	1.74
60 Hz requested	47–52	2.16	2.80	3.53	4.44	1.63	1.63
30 Hz, FF τ 0.15 s	29.8	1.37	1.73	2.19	2.31	1.18	1.19
60 Hz, FF τ 0.15 s	50.4	**1.27**	**1.60**	**2.03**	**2.27**	**1.05**	**1.05**

4.5 Harness-side skills and station tuning

Two failure modes on the unplug resisted wording: Dream-1 often closed the gripper on the way in, and it pulled sideways so the plug came half out. We added two narrow skills that act only through the station's existing move path. A *regrip* opens the gripper to 0.6, lowers 1.5 cm and closes when the gripper has shut on nothing; a *pull* lifts 6 cm and then 4 cm, 3 s per step. An opt-in station add-on applies an upward force of 20 N while the right gripper holds. All station changes are opt-in environment variables, refuse to load on any SDK other than the inspected version, and leave Dream's files unedited.

On a simulated Damiao bus running Dream's own driver, shortening the velocity feed-forward time constant from 0.8 s to 0.15 s cut joint tracking RMS by 40–50%, while doubling the bus tick barely changed it (Table 3). Requested 60 Hz ticks reached 47–52 Hz, because a late gripper reply holds each tick.

4.6 Joint fault detection

For monitoring the arms without brakes, we trained a per-joint fault detector on 16-reading windows (0.8 s at 20 Hz) of eight normalised features: position in range, margin to the nearer limit, velocity toward it, tracking error, commanded velocity toward the limit, and measured, commanded and differential torque. Faults (limit run, blocked joint, oscillation at 1.5–6 Hz, overload, overspeed) and near misses were injected into 413k samples of real policy motion from simulated matches, split by match file. Of 15 architectures, a dilated causal TCN [14] (32 channels, 4 layers, 10.5k weights) had the best timely recall, 0.987 ± 0.001 over three seeds. On the held-out file at a validation-chosen threshold it raised 0.2 false alarms per joint-hour at 0.91 timely recall, against 197 false alarms and 0.50 recall for the hand-written rules, whose limit-approach projection fires on every fast move toward a limit. Overspeed bursts of 1–3 readings are the exception (0.68 vs. 0.93), so both run together. All training motion is simulated YAM motion with injected faults; the numbers do not transfer to the Metal arms until it is retrained on recorded CAN traces.

5 A decision model for the harness

The rules above are hand-set constants. To tune them we wrote them as a small model in which today's rules are a special case. Let x_t be the stage, the time in it, the time left L, the attempt count and the evidence features $\phi(t)$ (judge verdict times confidence, judge progress, gripper transitions from `/arms`, detector boxes at the goal, the hub's plug count from a cable-end classifier).

Done evidence. A recursive log-odds score

$$\ell_t = \gamma\, \ell_{t-1} + \textstyle\sum_i w_i\, \phi_i(t), \qquad \text{done} \iff \ell_t \geq \theta_{\text{done}}(s). \tag{1}$$

The current rule ("two agreeing verdicts") is $\gamma = 1$ on agreement and 0 on disagreement, a single unit weight on the judge, and $\theta_{\text{done}} = 2$.

The pass-bar constraint. A false "done" on a pass bar leads the policy to touch the next stage's objects, which is a DQ. If each look has false-positive rate α and k independent looks are required, $P(\text{false done}) \approx \alpha^k$, and θ_{done} is set per stage so that this stays under a budget ε. Independence is the weak point. In the stress simulator, with two calls in flight, two false "cable out" verdicts arrived seconds apart and passed the two-look rule in 2 matches; the confirming looks shared the same frames. A minimum gap between confirming verdicts would restore independence.

Fail hazard and action choice. An attempt ends when $h_t = \sigma\big(b + w_f n_{\text{fail}} + w_r\,[p_{\max} - p_t - \delta]_+ + w_s(\tau - \tau_{\text{rise}})/\tau_{\text{stall}} + w_T\tau/\tau_{\text{out}} - w_{\text{on}}\mathbb{1}[\text{on target}]\big)$ exceeds θ_{fail}. The next action $a \in \{\text{continue, re-prompt, reset, home and retry, skip, abort}\}$ maximises

$$Q(a \mid x) = P_a(\text{success within } c_a)\, V_s + W(L - c_a) - \lambda\, R_a, \tag{2}$$

where V_s is the stage's points plus the value it unlocks, c_a the action's time cost, R_a its DQ or illegal-ball risk, and $W(L)$ the expected points of the remaining stages with L seconds left, computed backwards over the plan from each stage's success-versus-time curve $F_s(\tau)$. Continuing on stage s is worthwhile while

$$\frac{f_s(\tau)}{1 - F_s(\tau)}\, V_s \;\geq\; \frac{dW}{dL}, \tag{3}$$

which gives each stage a timeout that depends on its own success hazard and on how much of the match remains.

Tuning objective. $J(\theta) = \mathbb{E}[\text{points}] - \mu\, P(\text{DQ}) - \nu\, P(\text{illegal ball})$, estimated in the stress simulator with common random numbers and searched by coordinate ascent or CMA-ES [15]. A shadow-mode implementation logs what the model would do beside the live rules. We did not let it decide during the event, and the noise model of the simulator still needs calibration from labelled station frames before its optimum can be trusted.

6 Where the time goes

The station logged every policy call (Figure 3). The median round trip was 1,116 ms (p10 800, p90 2,733, max 8,655). Of that, model serving took 293 ms and the network 40 ms; the rest is gateway overhead that we cannot change. Each request uploads about 1.6 MB of camera images, so a slow uplink adds directly: on a stretch at 8.8 Mbit/s up, calls took 4.7 s and the arm did nothing useful.

Because the median sits on the pacer's 900 ms threshold, 67% of chunks (1,101 of 1,650) ran stop-and-wait. In that mode chunks arrive every 3.54 s (median) for 2.4 s of motion; in async mode every 1.55 s with no gap. The arms therefore stood still roughly 30% of the time. Two things we controlled made it worse: the harness's own 2 MB debug-view fetches slowed calls to 2–6 s until we switched to 35 KB single frames, and fresh starts sometimes returned first answers after 7–12 s that the client discarded, costing about 20 s per new session.

The harness adds its own delay at each stage boundary: a 3 s poll, a judge call of several seconds, then a confirming look. We estimate 5–8 s of idle robot per boundary, or 25–40 s of a 300 s match. A local estimator that fuses gripper and detector evidence (the ℓ_t above) could advance stages within about 0.5 s and keep the judge as a parallel veto; we designed but did not deploy it.

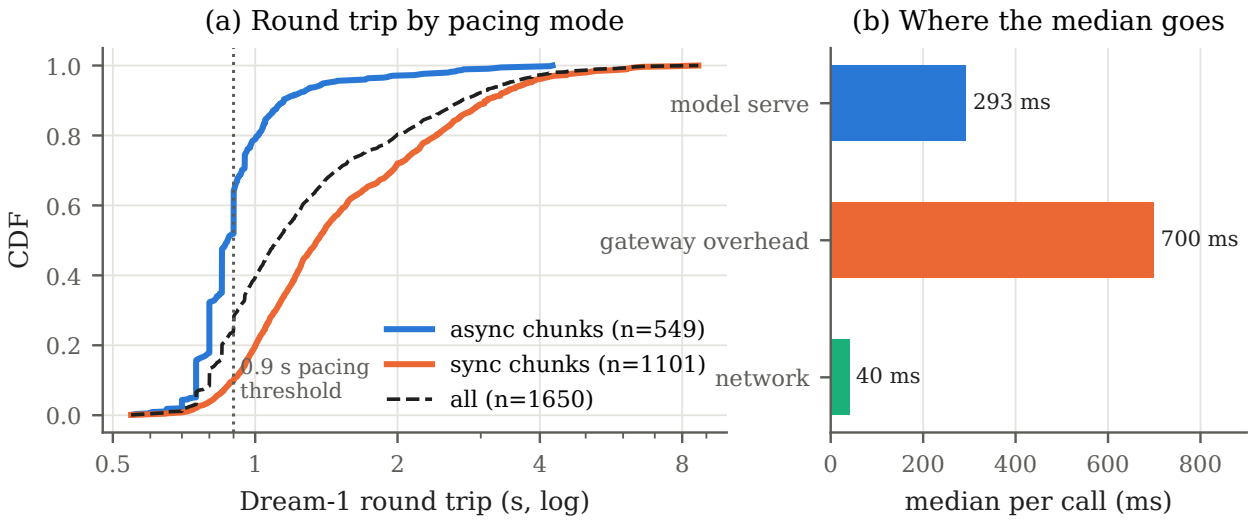


Figure 3: Dream-1 round trips on the station, 52 runs and 1,650 chunks between 18:06 on 7 Nov and 05:22 on 8 Nov (UTC). (a) Chunks requested in stop-and-wait (sync) mode have the slow tail; the pacer switches mode around a 0.9 s median. (b) Median components as logged; medians are not additive, so the parts do not sum to the 1.12 s median round trip.

Errors were frequent enough to plan for. In the same window there were 16 policy errors: 4 safety refusals ("unsafe instruction"; one triggered by a person's hand holding a plug), 4 `SVD did not converge` errors in the client IK, 4 empty action buffers (12 s without a chunk), and 4 replaced sessions. The harness now recovers from the first three without ending the match: refusals retry with the template's safety sentence, and IK errors trigger at most two automatic resets.

7 Simulators

Offline stress simulator. The real `Runner` and judge client run unchanged; everything around them is scripted. A virtual clock advances on every harness sleep, so a judge call returns only after its simulated latency and races between checks, verdicts and the policy happen as they would live. A table model tracks cables, gripper contents, the tower and the bin; each task prompt has a try time and a success probability, with switches for the failure modes we saw (the policy will not release the plug, picks the illegal ball, brushes a block during the unplug, knocks the tower). The station model injects faults with the exact error types and messages from the station log, plus frozen, empty, corrupt and lagged camera frames and judge API outages. A scripted judge answers from the table state at the newest frame's capture time. A referee scores rubric v4, including both DQ rules. A 5-minute match takes about 5 s, so a 37-scenario $\times$ 5-seed sweep fits on a laptop or our CPU cluster.

MuJoCo table with the real policy. A drop-in replacement for the station agent serves the same HTTP routes on Dream's MuJoCo YAM example [16], with a relay table (hub, cables, blocks, golf balls, bin) laid out from photographs and Dream's own control loop: camera windows, chunk requests to the live Dream-1 API, chunk validation and 20 Hz stepping. The harness cannot tell it from the station. The referee and an oracle verifier read object poses directly. It ran in Docker on a four-node home cluster and on cloud batch VMs. Two caveats apply: the arm is a YAM, not a Metal arm, so a Metal fine-tune cannot be evaluated here, and every match spends real policy credits.

8 Results

8.1 Harness robustness in the stress simulator

Fixes made overnight turned faults that used to end a match into small losses (Table 4). The most important was the plug: when the policy kept holding an unplugged plug, the old harness's next prompt led it to plug the cable back in,

Table 4: Points per seed in the stress simulator before and after the robustness fixes (4 seeds).

Scenario	Before	After
Happy path	87, 74, 90, 116	83, 94, 95, 95
Policy refuses on first start	0, 0, 0, 0 (match ended)	83, 94, 95, 95
Empty action buffer mid-run	10, 10, 15, 15 (ended)	79, 94, 95, 100
IK SVD failure	10, 0, 10, 10 (ended)	83, 84, 95, 95
12 s of corrupt frames	20, 15, 20, 20	83–90
Policy keeps the unplugged plug	plugged it back in, 4/4	never
Illegal ball picked on 30% of tries (6 seeds)	lost 4/6	lost 2/6 (detector guard)
Unplug brushes a block	DQ 4/4	DQ 4/4

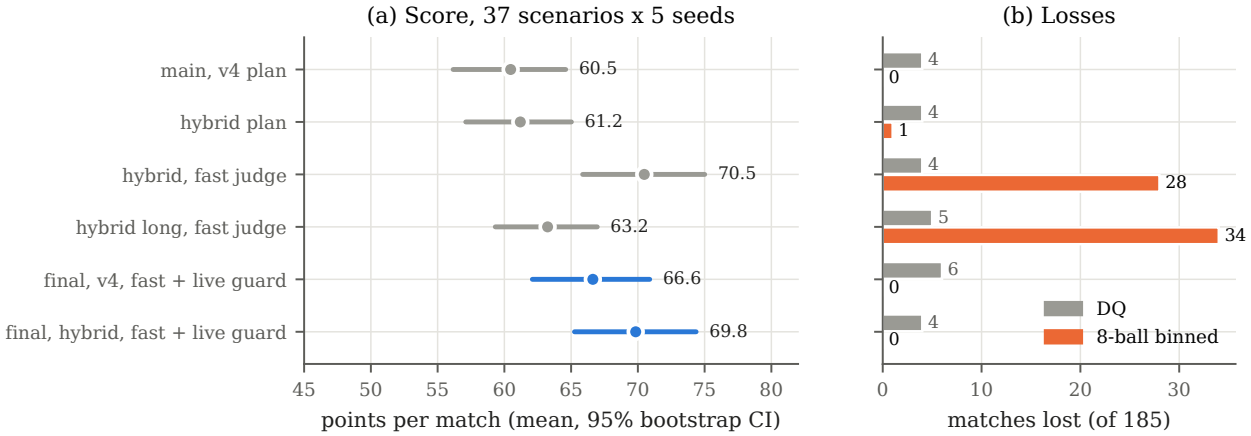


Figure 4: Stress simulator, 185 matches per configuration (37 scenarios × 5 seeds). "Fast" judge: 4–6 s per call; otherwise 6.5–13 s as measured for Haiku 5.5. Blue: configurations with the 10 Hz live guard and gripper overrides.

which the rubric forbids, in 4 of 4 seeds. The fixed harness never did.

Figure 4 compares whole configurations. Judge speed is the main lever on points, because a slow judge rarely leaves time for the ball stage: at the measured latency, configurations binned 0 to 0.09 balls per match on average. But a fast judge without the live guard reached the balls and lost 28 of 185 matches to the illegal ball (32 losses in total). With the guard and sensor overrides, the hybrid plan kept 69.8 points (95% CI 65.3–74.4) and lost none to the ball; the 4 remaining losses are DQs from the `reach_across` scenario, in which the policy brushes a block during the unplug, which no supervisor can prevent after the fact. A longer plan with more cable stages scored lower and lost 34 matches to the ball. Per-scenario means are in Appendix Figure 7.

Two plan changes were selected by sweeping at the measured judge latency (24 scenarios × 4 seeds): removing synchronous judge checks from stages that sensors can answer raised mean points from 54.9 to 60.0 with no new losses, and dropping the extra cable stages from the base-model plan was worth about 18 points per match. Under scenarios calibrated to base Dream-1's observed unplug rate (about 1 try in 6), the plan written for the base model (colour and position prompts) beat the plan built on the fine-tune's recorded task texts by about 20 points.

These numbers test the harness's logic against a scripted world. The simulator's unplug probability of 0.8 per try is far above what the real policy showed, and its judge never misjudges unless told to.

8.2 Judges

We benchmarked 30 judge configurations on the same 60 decisions (Figure 5). Stage-done questions were easy for most strong models; the guard question ("is a gripper holding the illegal ball?") separated them. GPT-6.1 Terra with no reasoning reached 98% done and 97% guard accuracy at a 2.8 s median; with low reasoning, 100% and 100% at 3.7 s, for about $0.07 per 60-decision match. Haiku 5.5 was perfect on done but 77–93% on the guard depending on reasoning level; Sonnet 5.5 through the CLI was perfect on both at 7.1 s. On a separate 120-decision run, Haiku 5.5 caught a held illegal ball only 55% of the time. The guard result is the reason the live guard does not depend on

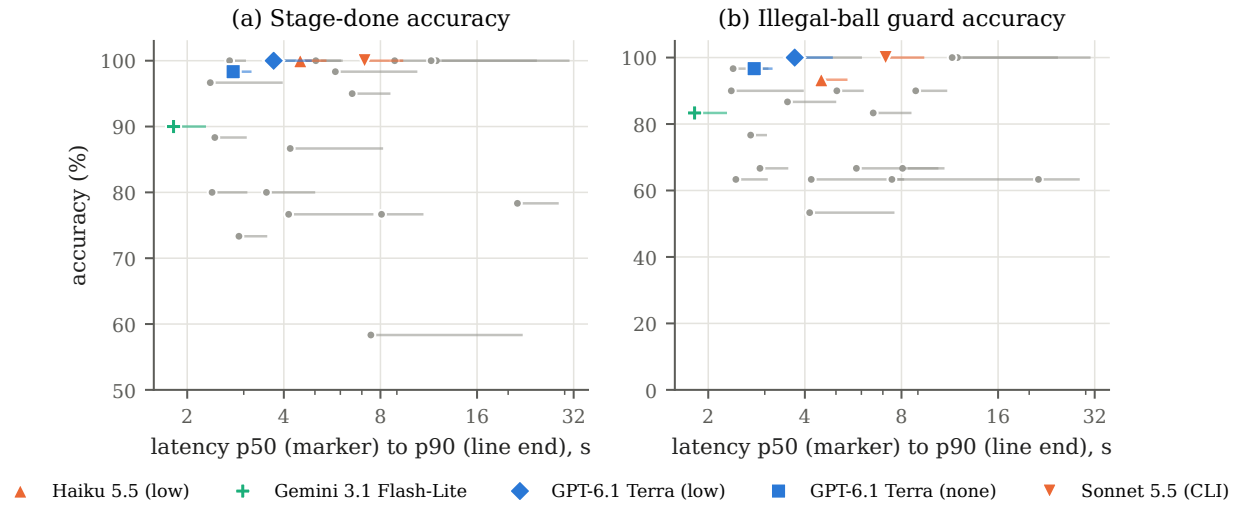


Figure 5: Judge benchmark: 30 model and reasoning-level configurations, 60 decisions each (30 guard questions) on the same frames, via OpenRouter or the Claude CLI. One configuration whose every call failed is omitted.

Table 5: Base Dream-1 through the harness on the MuJoCo relay table, 25 matches (one batch with a referee bug excluded).

Task	Result	Time
Unplug	a cable out in 14 of 25 matches	first unplug median 57 s (28–269 s)
Put the cable down	0 of 54 tries	policy keeps holding the plug
Stack	0 of 25 tries	–
Balls	never reached	–
DQs	3, all with the hub on the left	–

any judge. With 30 guard questions per configuration, a 100% score has a 95% lower bound of about 88%, so these rankings are coarse.

8.3 The real policy in MuJoCo

Table 5 shows base Dream-1 on the simulated table. It unplugged a cable in 14 of 25 matches, then kept holding the plug and never put it down in 54 tries, so no match reached the tower. All 3 DQs occurred with the hub placed to the left, where the plan's prompts ("the end of the hub nearest the right arm") no longer matched the scene. Later batches put the hub in front of the right arm, as on the station. In a separate 12-trial test, prompts naming an arm failed 0 of 4 times and the other arm still moved for 10–19 s; two goals in one prompt without naming arms completed one goal in 2 of 4 trials.

A cloud sweep meant to compare async pacing, operator notes and judge choice on this simulator (round 1, about 28 concurrent policy sessions) stopped when the account's inference credits ran out. The service reported this as an unauthorised, disabled key while account calls kept working. Only 10 of 28 matches were valid, scoring 0–42 points, and the comparison cannot be made from them. We now probe the policy with one call before every batch.

9 Teleoperation and fine-tuning

9.1 Two collection paths on the Metal arms

Leader/follower. One Metal arm becomes the leader, running MakerMods' gravity-compensated leader mode (k_p = 0, Pinocchio [17] gravity and friction feed-forward); the other follows. We subclass the event kit's collector so the dashboard, cameras, station lock, homing and LeRobot v2.1 writer are unchanged, and only the input differs. The first real episodes showed the follower sitting below its command by gravity over k_p: shoulder lift +4 to +6°, elbow +2.5

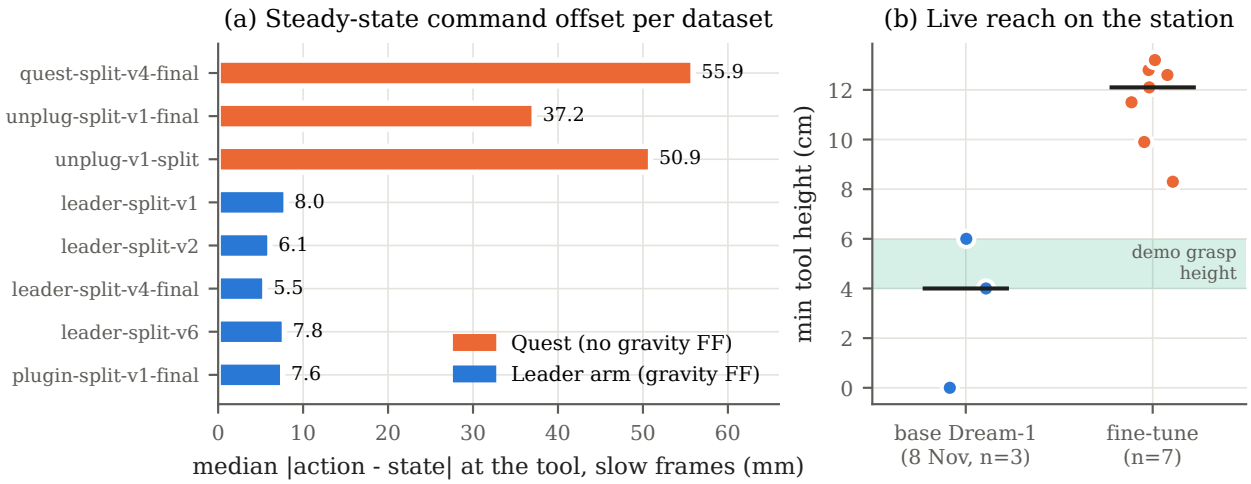


Figure 6: (a) Steady-state offset between commanded and measured tool position (frames with joint speed under 0.15 rad/s, so lag is excluded). Quest folders were recorded without gravity feed-forward. (b) Lowest tool height reached under the unplug prompt on the station, base model vs. fine-tune; bars are medians. The fine-tune never reached the height at which demonstrations grasp.

to +4.3°. Adding the leader's gravity model (which matches Dream's Metal gravity table to 0.01 N m) to the follower's MIT frames brought shoulder lift to +0.2°.

Quest headset. The kit's Quest path showed wrist flips: with joint 5 at its −82° limit margin, an 8–40° tool turn made the IK swing joints 4 and 6 by 130–190° over 1.5–2.5 s. We added heading calibration, a One Euro filter [18] on controller poses, a wrist-flip guard that holds tool orientation when the wrist turns a lot for little tool turn, a jump guard for tracking glitches, a 4 rad/s wrist cap and gravity feed-forward. Replaying 40 fast-wrist events from the recorded datasets through the IK, swings over 90° fell from 7 of 40 (worst 194°) to 2 of 40 (worst 116°) with tool position held within 8 mm. On leader-recorded motion with Quest-like noise, wrist lag fell from 105 ms to 27 ms and median turn error from 2.8° to 1.0°. These are offline replays; the new Quest launcher was not run with a headset during the event.

9.2 Why the fine-tune aimed high

The team fine-tuned Dream-1 on 170 episodes (48,413 frames) from ten folders: 1,000 steps, learning rate 10^{-5}, batch 32, end-effector actions, constant schedule, final loss 0.0034. On the station it was worse than the base model on every task. Under the unplug prompt its tool never went below 8–13 cm while reaching, and it closed the gripper at 11–16 cm; demonstrations grasp at 4–6 cm, and the base model reached 0–6 cm on the same day (Figure 6b).

The cause is in the data (Figure 6a). The 73 Quest episodes were recorded through the kit's follower, which sends zero torque, so each joint sagged below its target. Their actions sit about 5.5 cm (roughly 10° across joints 2–4) above the measured tool. The 97 leader episodes and the live station both apply gravity feed-forward, so command and arm agree to within 5–8 mm. The model was trained on two incompatible maps from state to action: in Quest episodes, holding the tool at height z takes a command at $z + 5.5$ cm. Live, that command is reached, the arm rises, and it is asked to rise again. Unplug and put-down are the tasks where both conventions appear (36 and 37 Quest episodes against 33 and 18 leader episodes).

We ruled out camera key mapping, view swaps, rotation, colour, resolution, camera position (under 2 px shift), joint order and units, gripper scaling, tool frame and schema differences across folders. Secondary causes were real but smaller: only 9 of 69 unplug demonstrations start within 0.3 rad of the home pose every live run starts from; all four block demonstrations are left-armed; playback at 20 Hz against 30 fps training adds about 80 ms of lag, about 1 cm at the arm's speeds.

To test whether a correction applied at run time could rescue the checkpoint, we built a MuJoCo model of one Metal arm with Dream's link table, gains and feed-forward, fitted the effective gravity load to the Quest sag, and validated it (replaying Quest actions without feed-forward reproduces recorded tool height with 10 mm median RMS). Replaying the seven live fine-tune runs' commands, no gravity scaling alone reached the 6 cm grasp bar; the only correction that did without passing through the table was a right-arm joint offset of 1.5 times the measured Quest sag, faded in with

reach (grasp height 5.5 cm, 1.8 cm margin above the table). This is an open-loop replay; the closed-loop policy would react to a lower arm. The data fix is simpler: replace each Quest action with the measured joints three frames later before computing end-effector labels, and screen every folder for steady-state action-minus-state offsets above about 1 cm before training. A corrected 500-step fine-tune takes about five hours end to end, which did not fit before the matches, so the final station plan ran base Dream-1.

10 What we could not measure

- **No physical success rates.** The station log records calls and errors but not outcomes, and the harness logs that record per-stage outcomes were not collected from the station before the event ended. Table 2 is judged by eye on a handful of trials. Human teleoperation times (unplug median 8.5 s over 15 demonstrations; put-down 4.8 s over 9) are the only real task timings we have.
- **Simulator fidelity.** The stress simulator's behaviour probabilities are assumptions, and its scenario mix is ours, so its absolute points are not predictions. The MuJoCo table uses a YAM arm, photo-estimated layout and an oracle verifier.
- **The real judge in the loop.** We benchmarked judges on fixed frames and ran the stress simulator with scripted verdicts, but did not run the real judge against the MuJoCo referee to measure false and late "done" rates.
- **Local detector colours** were tuned on phone photographs, not on station frames with a gripper closed around the white ball.
- **Fault detector** results are on simulated YAM motion with injected faults.
- **Tests.** The harness repository's last full run on its experimental branch was 407 passed, 4 failed, 5 skipped; two of the failures concern the async pacing test in the simulator.

11 Lessons

1. **Under gating rules, the supervisor's false positives are the main risk.** A false "done" on a pass bar costs the whole match, so confirmation must use independent evidence: different frames, a minimum gap, or a different sensor.
2. **Safety checks must not share a path with slow reasoning.** A 10 Hz colour detector with its own `/stop` made a fast judge safe; no judge we tested was reliable enough on the guard question to stand alone.
3. **Measure the serving loop before tuning prompts.** With a 0.9 s pacing threshold and a 1.1 s median round trip, request size and uplink decided whether the arms moved continuously. Our own debug traffic was part of the problem.
4. **Prompt a VLA by naming objects, not arms or motions.** Colour and position helped; arm names, direction adverbs and long coaching hurt. Motions the words could not fix (regrip, pull) were better done as small, separately tested skills.
5. **Check action-minus-state per data source before fine-tuning.** Two collection paths on the same arm differed by 5.5 cm in steady state, and the training loss gave no sign of it.
6. **Log outcomes, not just calls.** Without per-stage outcome logs on the station, a night of runs produced latency statistics but no success rate.

12 Conclusion

For a hosted VLA under a strict rubric, most of the engineering we could do sat outside the policy: when to declare a stage done, how fast to stop, what words to send, how to keep the serving loop in async mode, and whether the demonstration data agreed with the robot's control path. The harness, both simulators, the teleoperation tools and the analysis scripts are public. The next experiments are the ones the event did not leave time for: per-stage outcome logging on the station, the real judge against the MuJoCo referee, and a fine-tune on gravity-corrected data compared with the base model on the same seeded scenes.

Acknowledgments

We thank MakerMods and Dream Labs for the stations, Dream-1 access and the event kit, and Founders Inc for hosting Physical AI Hacks. Much of the code and analysis was written with AI coding assistants (Anthropic's Claude Code and OpenAI's Codex) under the authors' direction; the authors chose the experiments, ran the station, and checked the results reported here.

References

[1] Anthony Brohan, Noah Brown, Justice Carbajal, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

[2] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[3] Kevin Black, Noah Brown, Danny Driess, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[4] Michael Ahn, Anthony Brohan, Noah Brown, et al. Do as I can, not as I say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.

[5] Wenlong Huang, Fei Xia, Ted Xiao, et al. Inner monologue: Embodied reasoning through planning with language models. In *Conference on Robot Learning (CoRL)*, 2022.

[6] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.

[7] Yuqing Du, Ksenia Konyushkova, Misha Denil, et al. Vision-language models as success detectors. *arXiv preprint arXiv:2303.07280*, 2023.

[8] Zeyi Liu, Arpit Bahety, and Shuran Song. REFLECT: Summarizing robot experiences for failure explanation and correction. In *Conference on Robot Learning (CoRL)*, 2023.

[9] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems (RSS)*, 2023.

[10] Philipp Wu, Yide Shentu, Zhongke Yi, Xingyu Lin, and Pieter Abbeel. GELLO: A general, low-cost, and intuitive teleoperation framework for robot manipulators. *arXiv preprint arXiv:2309.13037*, 2023.

[11] Remi Cadene, Simon Alibert, Alexander Soare, et al. LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch. https://github.com/huggingface/lerobot, 2024.

[12] Xuanlin Li, Kyle Hsu, Jiayuan Gu, et al. Evaluating real-world robot manipulation policies in simulation. *arXiv preprint arXiv:2405.05941*, 2024.

[13] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[14] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.

[15] Nikolaus Hansen. The CMA evolution strategy: A tutorial. *arXiv preprint arXiv:1604.00772*, 2016.

[16] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.

[17] Justin Carpentier, Guilhem Saurel, Gabriele Buondonno, Joseph Mirabel, Florent Lamiraux, Olivier Stasse, and Nicolas Mansard. The Pinocchio C++ library: A fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives. In *IEEE/SICE International Symposium on System Integration (SII)*, 2019.

[18] Géry Casiez, Nicolas Roussel, and Daniel Vogel. 1€ filter: A simple speed-based low-pass filter for noisy input in interactive systems. In *ACM CHI Conference on Human Factors in Computing Systems*, 2012.

A Per-scenario results

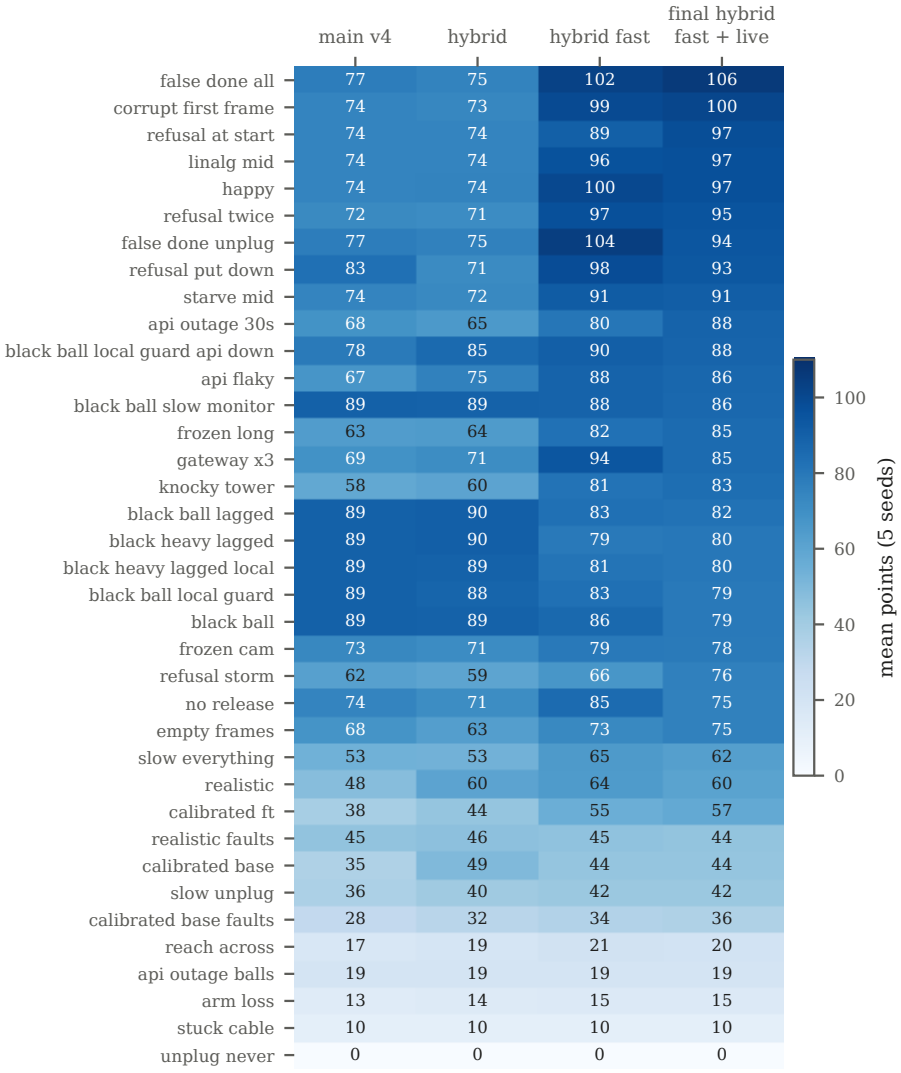


Figure 7: Mean points per stress scenario (5 seeds each), sorted by the final configuration. Scenarios include API outages, refusals, frozen and corrupt cameras, false "done" verdicts, an illegal ball near the bin, a knock-prone tower, and `unplug_never`, in which no configuration can score.

B Reproducing the numbers

- Stress sweep: `cluster/run_stress.sh 50 7 sweep1`, aggregated by `results/stress/final_20261108/analyze.py` in `task_simulation`.
- MuJoCo matches: `python sim_station.py -policy dream -scene hill_climb` with `experiments/sim_match.py`, or `cluster/run_matches.sh`; requires a Dream-1 key with credit.
- Station latency, stress and judge figures: `paper/make_figures.py` reads the logged files listed in its header.
- Quest replay bench: `quest/bench` in `follower_code` (branch `quest-continuity`).